

1. Scope

This platform is a patient engagement and hospital operations support system centered around:

- doctor/hospital knowledge graph search
- patient communication and reminders
- appointment and follow-up workflows
- lab and scan updates
- billing and insurance assistance
- FAQ support
- wellness engagement
- MCP server + KG retriever for tool-driven intelligence

2. High-level architectural style

Use an **event-driven microservices architecture** with a **channel-agnostic orchestration layer**.

Core style

- **API Gateway** for external access
- **Domain microservices** for each hospital capability
- **Event bus / message queue** for asynchronous workflows
- **Knowledge Graph** for doctor/hospital/entity retrieval
- **MCP Server** as a tool exposure layer for AI assistants / agents
- **Operational databases per service**
- **Audit + observability layer** for healthcare traceability

3. Layered LLD

3.1 Channel layer

These are the entry points for patients, staff, and bots:

- WhatsApp / SMS

- Mobile App
- Web Portal
- IVR / Voice bot
- Agent Console / Call center UI
- Internal Ops dashboard

All channels call the same backend through API Gateway.

3.2 Edge layer

API Gateway

Responsibilities:

- route requests to services
- auth token validation
- rate limiting
- request correlation ID
- tenant/hospital routing
- response aggregation for simple reads

Auth / Identity

Responsibilities:

- patient login
- staff RBAC
- service-to-service auth
- consent tracking
- session management

Recommended:

- OAuth2 / OIDC
- JWT for external requests
- mTLS for internal traffic

3.3 Orchestration layer

This layer manages multi-step business workflows.

A. Conversation Orchestrator

Handles:

- FAQ flows
- patient support conversations
- intent classification
- agent handoff
- calls MCP tools / KG retriever

B. Appointment Orchestrator

Handles:

- follow-up scheduling
- reschedule / cancel
- doctor slot selection
- appointment reminders

C. Notification Orchestrator

Handles:

- medication reminders
- lab reminders
- scan reminders
- waiting time updates
- lab result status messages
- wellness tips

D. Care Journey Orchestrator

Handles:

- post-visit follow-up
- symptom check-in
- recovery tracking

- escalation to nurse/doctor if risk thresholds hit

E. Billing Support Orchestrator

Handles:

- estimate lookup
- insurance document checklist
- claim status queries
- payment link workflow

F. Feedback Orchestrator

Handles:

- patient feedback
- NPS/CSAT capture
- issue ticket creation for low ratings

3.4 Domain services

Each service owns its own business logic and storage.

1. Doctor & Hospital Graph Search Service

Purpose:

- graph-based search on doctors, specialties, departments, hospital branches, facilities

Functions:

- search doctor by specialty
- find hospital/branch with service availability
- find doctor by language, experience, availability, insurance coverage
- related-entity traversal

Storage:

- Graph DB

2. KG Construction Service

Purpose:

- builds and updates hospital knowledge graph

Inputs:

- doctor master data
- hospital departments
- branch/facility data
- schedules
- insurance mapping
- FAQ corpus
- treatment/service taxonomy

Pipeline:

- ingest -> normalize -> deduplicate -> entity resolution -> graph edges -> version publish

3. Queue & Waiting Time Service

Purpose:

- live OPD / diagnostic queue status

Functions:

- token/queue lookup
- estimated wait time
- doctor delay updates
- room/counter status
- push alerts when nearing turn

Dependencies:

- HIS / queue system
- appointment system

4. Follow-up Appointment Service

Purpose:

- auto/manual follow-up scheduling after visit/discharge/test result

Functions:

- recommended follow-up window
- slot suggestions
- booking / rescheduling
- reminders
- no-show handling

Dependencies:

- doctor schedules
- visit summary
- care plan

5. Prescription Medication Reminder Service

Purpose:

- reminder generation from e-prescriptions

Functions:

- dosage schedule creation
- reminder notifications
- adherence tracking
- missed-dose follow-up
- refill reminder

Dependencies:

- ePrescription / EMR

6. Lab Test & Scan Reminder Service

Purpose:

- remind patients to complete ordered tests/scans

Functions:

- order reminder
- fasting/preparation instruction
- due date reminder
- branch/location suggestion
- missed test follow-up

Dependencies:

- LIS / RIS orders
- appointment availability

7. Lab Test Status Update Service

Purpose:

- communicate test progress

Functions:

- sample collected
- in processing
- report ready
- delayed / action needed
- download/view report link

Dependencies:

- LIS / RIS / PACS

8. Post-Visit Follow-up Service

Purpose:

- check patient well-being after visit/discharge/procedure

Functions:

- symptom questionnaire

- recovery checklist
- red-flag detection
- escalation workflow
- nurse call task creation

Dependencies:

- EMR encounter/discharge data
- triage rules engine

9. Patient Feedback Collection Service

Purpose:

- collect structured and unstructured feedback

Functions:

- NPS/CSAT
- doctor experience feedback
- facility feedback
- complaint categorization
- ticket creation for issues

Dependencies:

- CRM / ticketing

10. Billing & Insurance Assistance Service

Purpose:

- help with payment and claims support

Functions:

- cost estimate lookup
- bill explanation
- insurance eligibility guidance
- pre-auth document checklist
- claim status support

- outstanding bill reminders

Dependencies:

- billing system
- insurance aggregator / TPA system

11. FAQ 24x7 Support Service

Purpose:

- always-on support chatbot

Functions:

- FAQs on doctors, timings, departments, procedures, insurance, reports, billing
- fallback to human agent
- calls KG retriever + FAQ search

Dependencies:

- vector store / knowledge base
- graph retriever
- MCP tools

12. Health Tips & Wellness Reminder Service

Purpose:

- preventive care and engagement

Functions:

- wellness nudges
- vaccination reminders
- chronic care reminders
- seasonal health campaigns
- personalized health tips

Dependencies:

- profile/preferences

- care segments / cohorts

13. MCP Server

Purpose:

- standardized tool layer for AI assistants and external agent runtimes

Exposed tools:

- search_doctors
- search_hospitals
- get_wait_time
- book_followup
- get_lab_status
- send_medication_reminder
- collect_feedback
- billing_help
- faq_query
- retrieve_kg_context

This should be the **single tool invocation surface** for LLM/agent workflows.

14. KG Retriever Service

Purpose:

- retrieval service over graph + documents

Responsibilities:

- graph traversal
- hybrid retrieval: graph + keyword + vector
- entity linking from user query
- evidence packaging for AI responses

Input:

- natural language query / structured intent

Output:

- ranked entities
- relations
- citations/evidence snippets
- tool-ready structured payload

4. Suggested service-to-database mapping

Service	Primary DB
Auth	PostgreSQL
Appointment Service	PostgreSQL
Queue Service	Redis + PostgreSQL
Medication Reminder	PostgreSQL
Lab Reminder / Status	PostgreSQL
Feedback Service	PostgreSQL
Billing Support	PostgreSQL
FAQ Support	Vector DB + PostgreSQL
KG Construction / Graph	Neo4j / Amazon
Search	Neptune
Notifications	PostgreSQL + Redis
Audit / Events	Kafka + object storage
Analytics	Data warehouse

Recommended infra choices

- **OLTP:** PostgreSQL
- **Cache / short-lived queue states:** Redis
- **Graph:** Neo4j
- **Event bus:** Kafka / RabbitMQ
- **Search:** OpenSearch / Elasticsearch
- **Vector store:** pgvector / Pinecone / Weaviate
- **Object storage:** S3-compatible store for reports, logs, exports

5. Core data model

5.1 Key entities

Patient

- patient_id
- mrn
- name
- dob
- gender
- phone
- email
- consent_flags
- preferred_language
- communication_preferences
- insurance_provider_id
- policy_number

Doctor

- doctor_id
- name
- specialty
- subspecialty
- qualifications
- languages
- years_experience
- branch_id
- consultation_modes
- schedules
- insurance_accepted

Hospital / Branch

- hospital_id
- branch_id
- address
- geo_location

- specialties
- diagnostic_capabilities
- timings
- insurance_supported

Appointment

- appointment_id
- patient_id
- doctor_id
- branch_id
- appointment_type
- scheduled_at
- status
- followup_of_visit_id
- queue_token

Prescription

- prescription_id
- patient_id
- visit_id
- medication_name
- dosage
- frequency
- duration
- reminder_schedule

Lab Order

- lab_order_id
- patient_id
- visit_id
- order_type
- test_name
- prep_instruction
- scheduled_at
- status
- report_url

Feedback

- feedback_id
- patient_id
- encounter_id
- rating
- category
- comments
- escalation_status

Billing Case

- billing_case_id
- patient_id
- visit_id
- estimate_amount
- billed_amount
- insurance_status
- preauth_status
- claim_status

5.2 Knowledge graph schema

Nodes

- Patient
- Doctor
- Specialty
- Department
- Hospital
- Branch
- Service
- Procedure
- InsuranceProvider
- DiagnosticTest
- Scan
- FAQTopic
- AppointmentSlot

Relationships

- DOCTOR_BELONGS_TO_BRANCH
- DOCTOR_HAS_SPECIALTY
- BRANCH_OFFERS_SERVICE
- BRANCH_ACCEPTS_INSURANCE
- DOCTOR_AVAILABLE_AT_SLOT
- PROCEDURE_REQUIRES_TEST
- FAQ_RELATES_TO_SERVICE
- DOCTOR_SPEAKS_LANGUAGE
- TEST_AVAILABLE_AT_BRANCH

Example graph query:

- “Find cardiologists in Bangalore branch accepting Insurance X and available tomorrow evening”

6. API design

6.1 External APIs

Doctor / hospital search

POST /api/v1/search/doctors

```
{
  "specialty": "cardiology",
  "location": "Bangalore",
  "language": "English",
  "insurance": "Star Health",
  "availability_date": "2026-08-18"
}
```

Queue status

GET /api/v1/queue/{appointment_id}

Response:

```
{  
  "appointment_id": "APT123",  
  "queue_token": "Q45",  
  "patients_ahead": 6,  
  "estimated_wait_minutes": 28,  
  "doctor_status": "running_late"  
}
```

Follow-up booking

POST /api/v1/appointments/followup

Medication reminders

POST /api/v1/reminders/medication/create

Lab status

GET /api/v1/labs/{lab_order_id}/status

Feedback submit

POST /api/v1/feedback

Billing help

POST /api/v1/billing/support/query

FAQ ask

POST /api/v1/support/faq

6.2 Internal service APIs

Prefer:

- REST for synchronous CRUD/query
- gRPC for low-latency internal service calls
- events for async workflow updates

Examples:

- AppointmentCreated
- QueueUpdated
- PrescriptionIssued
- LabOrderCreated
- LabStatusChanged
- FeedbackSubmitted
- BillingCaseUpdated

7. Event-driven workflows

7.1 Prescription reminder flow

1. EMR publishes PrescriptionIssued
2. Medication Reminder Service consumes event
3. Builds schedule
4. Stores reminder plan
5. Notification Orchestrator sends reminders through preferred channel
6. Patient response updates adherence status

Event payload

```
{
  "event_type": "PrescriptionIssued",
  "patient_id": "P1001",
  "visit_id": "V3001",
  "prescription_id": "RX9001",
  "medications": [
    {
      "name": "Metformin",
      "dosage": "500mg",
      "frequency": "twice_daily",
      "duration_days": 30
    }
  ]
}
```

7.2 Lab status update flow

1. Lab order created in LIS
2. LabOrderCreated event published
3. Reminder service sends pre-test instructions
4. LIS updates status to collected / processing / ready
5. LabStatusChanged event published
6. Notification Orchestrator pushes update to patient

7.3 Post-visit follow-up flow

1. Encounter closed in EMR
2. VisitCompleted event published
3. Post-Visit Service determines follow-up template by visit type
4. Sends check-in after configured interval
5. If red-flag symptom -> escalation ticket -> nurse/doctor alert

8. MCP server design

The MCP server should act as a **tool adapter layer** for AI assistants.

8.1 MCP tools

- doctor_search
- hospital_search
- queue_status_lookup
- followup_schedule
- medication_reminder_create
- lab_reminder_create
- lab_status_lookup
- post_visit_followup_start
- feedback_submit
- billing_support_query
- faq_support_query
- kg_retrieve_context

8.2 MCP architecture

- MCP Server
 - Tool registry
 - Auth context mapper
 - Rate limiter
 - Audit logger
 - Response normalizer

Example tool contract

```
{
  "tool": "kg_retrieve_context",
  "input": {
    "query": "Find orthopedic doctors in Hyderabad accepting cashless insurance"
  }
}
```

Output:

```
{
  "entities": [
    {"type": "Doctor", "id": "D12", "name": "Dr. Rao"},
    {"type": "Branch", "id": "B7", "name": "Hyderabad Central"}
  ],
  "relations": [
    "DOCTOR_HAS_SPECIALTY",
    "BRANCH_ACCEPTS_INSURANCE"
  ],
  "evidence": [
    "Dr. Rao is mapped to Orthopedics",
    "Hyderabad Central supports cashless claims"
  ]
}
```

9. KG retriever design

Use a **hybrid retriever**:

Retrieval stages

1. query understanding
2. entity extraction
3. graph traversal
4. vector retrieval from FAQs/docs
5. ranking + deduplication
6. response evidence packaging

Components

- Entity linker
- Graph query builder
- Vector retriever
- Ranker
- Context assembler

Why hybrid

Graph search is strong for:

- relationships
- filtered discovery
- multi-hop queries

Vector/FAQ retrieval is strong for:

- policy text
- instructions
- natural language answers

Use both together.

10. Queue and scheduler internals

10.1 Notification scheduling

Use a scheduler service with:

- cron-based campaign jobs
- event-triggered reminders
- retry policy
- dead-letter queue
- per-channel throttling

Tables

- notification_template
- notification_job
- notification_delivery
- patient_channel_preference

10.2 Waiting time computation

Inputs:

- doctor actual start time
- average consultation duration
- patients ahead
- emergency interruptions
- no-show rate

Formula:

estimated_wait =
(patients_ahead * avg_consultation_minutes)
+ delay_adjustment
- no_show_adjustment

Keep latest state in Redis for fast reads.

11. Security and compliance

For healthcare systems, this is critical.

Required controls

- encryption in transit and at rest
- RBAC + ABAC for staff access
- patient consent management
- audit trails for every data access
- PHI masking in logs
- secure secrets management
- environment isolation
- data retention policies
- incident monitoring

Strong recommendation

Split data into:

- PHI / clinical data
- operational metadata
- analytics copies

Avoid exposing raw PHI to FAQ/chat layers unless explicitly required and authorized.

12. Non-functional requirements

Performance

- search API < 500 ms for common queries
- queue lookup < 200 ms
- notification enqueue < 100 ms
- lab status propagation < 1 min after source update

Availability

- 99.9% for support channels
- graceful degradation if KG/vector search unavailable

Scalability

- horizontally scale orchestration and notification services
- partition events by hospital_id / branch_id

Reliability

- idempotent consumers
- retry with DLQ
- saga/compensation for multi-step operations

Observability

- distributed tracing
- business metrics
- audit dashboards
- alerting on failed notifications, stale lab statuses, queue sync lag

13. Recommended module boundaries

A practical deployment grouping:

Core platform

- API Gateway
- Auth
- User Profile / Consent
- Notification Service
- Audit Service

Care workflow domain

- Appointment Service
- Follow-up Service
- Medication Reminder Service
- Lab Reminder / Status Service
- Post-Visit Service

Patient engagement domain

- FAQ Support
- Feedback Service
- Wellness Service
- Billing Assistance

Knowledge domain

- KG Construction
- Graph Search
- KG Retriever
- MCP Server

Integration domain

- HIS/EMR Connector
- LIS/RIS Connector
- Billing/Insurance Connector
- Messaging Provider Connector

14. Suggested integration adapters

Hospital systems

- HIS / EMR
- scheduling system
- doctor roster
- discharge summary feed

Diagnostics

- LIS
- RIS
- PACS

Finance

- billing engine

- payment gateway
- TPA / insurance API

Communication

- SMS gateway
- WhatsApp Business API
- email provider
- push notification provider
- IVR provider

All adapters should publish normalized internal events so domain services stay decoupled from vendor-specific schemas.

15. Suggested LLD sequence example

Use case: “Patient asks for follow-up after lab result”

1. Patient messages support bot
2. Conversation Orchestrator detects intent
3. MCP server invokes lab_status_lookup
4. If report ready, KG retriever fetches doctor + specialty + follow-up policy
5. Appointment Orchestrator queries slots
6. Bot returns top 3 follow-up slots
7. Patient confirms
8. Appointment Service books slot
9. Notification Orchestrator sends confirmation
10. Audit log written

16. Minimal deployment topology

- 2x API Gateway instances
- 2x Conversation Orchestrator
- 2x Appointment Orchestrator
- 2x Notification workers
- 2x Queue service nodes
- 2x FAQ/KG retriever nodes

- 3-node Kafka cluster
- PostgreSQL HA
- Redis cluster
- Neo4j cluster
- object storage
- centralized logging + tracing

Kubernetes is a good fit if this will scale across hospital groups.

17. Recommended folder/service structure

```

hospital-platform/
  api-gateway/
  auth-service/
  patient-profile-service/
  appointment-service/
  queue-service/
  medication-reminder-service/
  lab-service/
  post-visit-service/
  feedback-service/
  billing-support-service/
  faq-support-service/
  wellness-service/
  kg-construction-service/
  graph-search-service/
  kg-retriever-service/
  mcp-server/
  notification-service/
  audit-service/
  integrations/
    his-connector/
    lis-connector/
    ris-connector/
    billing-connector/
    messaging-connector/

```

18. Best implementation recommendation

If you want this to be maintainable, I'd recommend:

- **PostgreSQL** for transactional domains
- **Neo4j** for doctor/hospital KG
- **Kafka** for hospital workflow events
- **Redis** for queue state + short-term caching
- **OpenSearch** for fast keyword search
- **pgvector / vector DB** for FAQ and semantic retrieval
- **MCP Server** as the only AI-tool-facing access layer
- **Hybrid KG + vector retriever** for intelligent support

19. Final architecture summary

The cleanest LLD is:

- channels at the top
- API gateway + auth at the edge
- orchestration layer for patient journeys
- domain microservices for each hospital workflow
- event bus for async coordination
- graph DB + KG retriever for intelligent search/support
- MCP server as the AI tool interface
- hospital integrations on the right
- operational databases and audit below

This gives you:

- modularity
- easy feature expansion
- real-time updates
- AI readiness
- hospital-system integration safety